

Q.1. Explain iterative Algorithm Design Issues. How Redundant Computation can be removed from below example:

$X = 0$

for $I = 1$ to N do

{

$X = X + 1;$

$Y = (a * a * a + c) * X * X;$

Print (y)

}

Iterative Algorithm Design Issues:-

1) The iteration may be explicit through a looping construct or it can be due to a recursive algorithm.

2) Even a small amount of excess time spent within the loop can lead to a major loss in the efficiency of amount of access time spent within the loop can lead to a major loss is the efficiency of algorithm.

3) The situation can even be worse if there are loops within loops or nested algorithm!

Design issues of the iterative algorithms:-

1) Iterations using the loop control structure

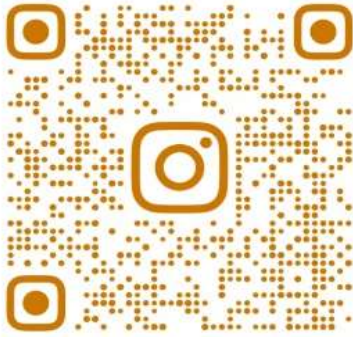
2) Improving the efficiency of the algorithms

3) Estimation of time & space requirements

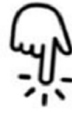
4) Expressing the complexities using the order notations.

5) Applying different algorithmic strategies.

Join community by clicking below links



@SPPU_ENGINEERING_UPDATE

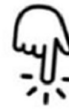


Insta Page

https://www.instagram.com/sppu_engineering_update



@SPPU_TE_BE_COMP



Telegram Channel

https://t.me/SPPU_TE_BE_COMP



WhatsApp Channel

<https://whatsapp.com/channel/0029VaAhRMdAzNbmPG0jyq2x>

① Iterations using loop control structure:

For any loop we specify -

1. Initial condition - that needs to be true before the loop begins execution.
2. The Invariant relation that must hold before during & after each iteration of the loop.
3. The condition under which the loop must be terminate.

e.g.

$i \rightarrow 0$

$Sum \rightarrow 0$

for while ($i \leq n$)

$i \rightarrow i+1$

$Sum \rightarrow Sum + a[i]$

end while.

② Improving efficiency of algorithm:-

Efficiency in terms of Time & Space techniques to improve algo. efficiency -

1. Removing redundant computations outside loops:

The most common mistake when using loops is to repeatedly re-calculate part of an expression that remains constant throughout execution phase of entire loop.

e.g. $x = 0$

for $i = 1$ to N do

{

$x = x + 1;$

$y = (a * a * a + c) * x * x;$

} print (y);

How we can remove unnecessary multiplications & computations:-

```

a1 = a * a * a + c;
x = 0
for i = 1 to N do
{
    x = x + 1;
    y = a1 * x * x;
    print (y)
}
    
```

Q.2. Write short note on Algorithm as a technology.

-
- ① Computing time & memory space are limited resources, so we should use them sensibly.
 - ② It is achieved by the usage of efficient algorithms that need less space & time.
 - ③ Different algo. take diffⁿ time to solve the same problem.
 - ④ Consider a faster computer A (executes 10^{10} instruction / second)
 - ⑤ Slower Computer B (executes 10^7 instruction / second)
 - ⑥ Let, insertion sort's running time is $C_1 \cdot n^2$ & merge sort is running time as $C_2 \cdot n \cdot \log n$.
 - ⑦ Suppose C_1 is 2 & C_2 is 50.
 Faster (A) computer running Insertion sort.
 Slower (B) computer running Merge sort.

A is 1000 times faster than B.

To sort 10 million numbers A takes $\rightarrow$

$$= \frac{2 \cdot (10^7)^2 \text{ instructions}}{10^{10} \text{ instructions/second}}$$

$$= 20,000 \text{ seconds}$$

B takes $\rightarrow$

$$= \frac{50 \cdot 10^7 \cdot \log 10^7 \text{ instructions}}{10^7 \text{ instruction/second}}$$

$$\approx 1163 \text{ seconds}$$

By using an algorithm whose running time grows more slowly even with a poor compiler, computer B runs more than 17 times faster than computer A.

⑧ Above eg. shows that we should consider algo. like computer hardware as a technology.

⑨ Fast hardware algo. are as important as—

- Advanced computer architecture
- Object oriented system
- Fast networking

Q.3. How to confirm correctness of algorithm also explain with example.

→ There are 2 methods of correctness of an algo. -

- 1) Using loop invariants
- 2) By mathematical inductions.

① Loop Invariants:-

3 things of loop invariants:

1. Initialization - It is true prior to the first iteration of the loop.

2. Maintenance - If it is true before an iteration of the loop, it remains true before the next iteration.

3. Termination - When the loop terminates, the invariant given us a useful property that helps show that the algo. is correct.

Example:-

Insertion-sort (A)

for $j = 2$ to $A.length$

$key = A[j]$

$i = j - 1$

 while $i > 0$ and $A[i] > key$

$A[i+1] = A[i]$

$i = i - 1$

$A[i+1] = key$

Initialization →

- We start by showing that loop invariant holds before first loop iteration, when $j=2$. The subarray $A[1..j-1]$ consists of just single element $A[1]$, which is sorted.
- Which shows that loop invariant holds prior to the first iteration of the loop.

Maintenance →

- Showing that each iteration maintains the loop invariants.
- For loop works by moving $A[j-1], A[j-2], A[j-3], \dots$ & so on by one position to right until it finds proper position for $A[j]$, ~~for~~ at which points it inserts value of $A[j]$.
- The subarray $A[1..j]$ then consists of elements originally in $A[1..j]$ but in sorted order.

Termination →

- Finally we examine what happens when the loop terminates.
- For loop terminates when $j > A.length$ means $j > n$ i.e. $j = n+1$, in that case subarray $A[1..n]$ consists of elements originally in $A[1..n]$ but in the sorted order.
- So, we conclude that the entire array is the sorted.

Hence, algo is correct.

or

② Correctness using mathematic induction $\rightarrow$

3 steps -

1. Basis of induction
2. Inductive Hypothesis
3. Inductive step.

Example -

Sum of the first n positive integer equal to $n(n+1)/2$

Theorem $\rightarrow 1+2+3+\dots+n = n(n+1)/2$

Proof $\rightarrow$

① Basis of induction:

Since,

$$S(1) = 1 = 1(1+1)/2 = 2/2 = \underline{\underline{1}}$$

$\therefore$ Formula is true for $n=1$.

② Inductive Hypothesis:

Assume that proof is true for $n=k$ i.e.

$$S(k) = 1+2+\dots+k = k(k+1)/2 \quad \text{--- ①}$$

③ Inductive step:

Now, show that formula is true for $n=k+1$.

$$\text{i.e. } S(k+1) = 1+2+\dots+k+k+1 = S(k) + S(k+1)$$

From ①,

$$\therefore S(k+1) = k(k+1)/2 + (k+1)$$

$$= \frac{k}{2}(k+1) + 1(k+1)$$

$$= \left(\frac{k+1}{2}\right)(k+1)$$

$$= \frac{(k+2)}{2}(k+1) = \frac{(k+1)(k+2)}{2}$$

$\therefore$ Formula is true for $k+1$.

Q.4 How do we analyze algo., Explain what is meant by space & time complexity. Find frequency count of -

	Frequency	total
Void fun()	—	
{	—	
Int a;	—	
a=0;	1	1
For (i=0; i<n; i++)	$i=1 \rightarrow 1$ $i<n \rightarrow n+1$ $i++ \rightarrow n$	$n+1$ n
{		
a=a+i;	n	n
}		
Print ("%d", a);	1	1
}		

Analysis of Algorithm:-

- We can compare one algorithm with another & choose the best. This is called as analysis of algorithm.
- Analysis involves measuring the performance of an algo.

Time Complexity →

The amount of time taken by algo. to perform the intended task.

Space Complexity →

The amount of memory need to perform the task.

The space complexity is computed by considering the data & their sizes.

① Time Complexity →

- The maximum time req^d for execution of program as a funⁿ of the i/p size
- Time complexity is time taken by program & sum of the compile & execution time.
- We can determine the no. of steps needed by program to solve particular problem instance in one of 2 ways -

① By introducing one variable, as global & initializing it as zero & increment it by each statement in program.

② Compute no. of times each statement will be executed manually.

② Space Complexity →

- Space complexity specifies the amount of computer memory req^d during program execution as a function of i/p size
- Space complexity measures 2 different times -

① Compile time space complexity -

- Storage req^d of a program at compile time.
- It is determined by summing up the storage size of each variable.

② Runtime Space Complexity -

- If program is recursive or uses dynamic variables or dynamic data structure; then there is need to determine runtime space complexity.
- The memory requirement is summation of the program, space, data space & stack space.

e.g.

Time complexity $\rightarrow 3n + 4$
 $= O(n)$

Space Complexity $\rightarrow 3$ (constant)

Memory to store $\rightarrow$

3 variables $\rightarrow$ a, n & i

Q.5. Explain the various algorithm design methodology to solve a problem.
 $\rightarrow$

① Divide & Conquer $\rightarrow$

Solving a large problem by breaking the problem into smaller sub-problems, solving the sub-problems & combining the soln.
e.g. binary search, Merge sort.

② Greedy $\rightarrow$

Solving problem by selecting the best option available from given options.
e.g. Fractional knapsack problem, Job sequencing.

③ Dynamic $\rightarrow$

Solving problem by breaking them down into a collection of simplest subproblems solving each of those subproblems just once, and storing their soln for future use.
e.g. 0/1 Knapsack, OBST.

④ Branch & Bound $\rightarrow$

Total set of solⁿ can be partitioned into smaller subsets of solⁿ.

e.g. TSP

⑤ Backtracking $\rightarrow$

It uses recursive calling to find solⁿ by building a solⁿ step by step increasing values with time.

e.g. N-Queens Problem.

Q6 Why Correctness of algorithm is essential. Explain correctness of Algorithm with mathematical induction example.

→ Why correctness of algorithm is essential?

1. For any problem, we first get its correct solution by performing any valid logical computations
2. once the problem is cracked then only we can think of better solution to solve the same problem in a more efficient way
3. if an algorithm is incorrect then the efforts for improving its efficiency will be in vain.
4. before improving the efficiency of any algorithms we confirm its correctness. so correctness of algorithm is essential.

correctness of algorithm with mathematical induction example.

Example :- Sum of first n positive integers = $n(n+1)/2$

theorem :- $1 + 2 + \dots + n = \frac{n(n+1)}{2}$

proof :-

step 1 :- Basis of induction

$$\begin{aligned}\text{Since } S(1) &= 1 \\ &= n(n+1)/2 \\ &= 1(1+1)/2 \\ &= 1\end{aligned}$$

∴ this Formulae is Right for $S=n=1$

step 2 : Inductive step

Assume that proof is true for $n=k$ that means

$$S(k) = 1 + 2 + \dots + k = k(k+1)/2$$

step 3 : inductive step

now show that formula is true for $n=k+1$

$$S(k+1) = 1 + 2 + \dots + k + (k+1) = S(k) + (k+1)$$

$$\text{Since } S(k) = \frac{k(k+1)}{2}$$

By inductive hypothesis step.

$$\text{Hence } S(k+1) = k(k+1)/2 + (k+1)$$

$$= \frac{k(k+1)}{2} + 1(k+1)$$

$$= (k+1) \left[\frac{k+1}{2} \right]$$

$$= (k+1) \frac{(k+2)}{2}$$

so the formulae is true for $k+1$ also.

Q7 differentiate between P class and NP class problems.

→

	P class	NP class
1.	P problems are set of problems which can be solved in polynomial time by deterministic algorithm	NP Problems are the problems which can be solved in non-deterministic polynomial time.
2.	P problems can be solved and verified in polynomial time	Solutions to NP problems cannot be solved in polynomial time but if solution is given it can be verified in polynomial time.
3.	A problem is subset of NP problems	NP problems are superset of P problems.
4.	All P problems are deterministic in nature	All NP problems are non-deterministic in nature.
5.	Example:- Selection sort Linear search	Example:- TSP Knapsack problem

Q8 prove that 3-SAT problem is NP complete?

→

Proof:

1) 3-sat is NP. Given an assignment we can just check that each clause is covered.

2) 3-sat is hard. To prove that a reduction from sat to 3-sat must be provided.

- we will transfer each clause independently based on its length.

- Reduction From SAT to 3-SAT

• suppose a clause contains K literals.

① If $K=1$ (meaning $c_i = \{z_i\}$) we can add 2 new variables v_1 and v_2 and transform this into 4 clauses

$$\{v_1, v_2, z_i\} \quad \{v_1, \bar{v}_2, z_i\} \quad \{\bar{v}_1, v_2, z_i\} \quad \{\bar{v}_1, \bar{v}_2, z_i\}$$

② If $K=2$ ($c_i = \{z_1, z_2\}$) we can add in one variable v_1 and 2 new clauses.

$$\{v_1, z_1, v_2\} \quad \{\bar{v}_2, z_1, z_2\}$$

③ if $K=3$ ($c_i = \{z_1, z_2, z_3\}$) we will move the clause as it is

④ if $K>3$ ($c_i = \{z_1, z_2, \dots, z_K\}$) we can add in $K-3$ new variables ($v_1, \dots, v_{K-3}$) and $K-2$ clauses.

$\{z_1, z_2, v_1\} \quad \{ \overline{v_1}, z_3, v_2 \} \quad \{ \overline{v_2}, z_4, v_3 \}$

--- $\{ \overline{v_{k-3}}, z_{k-1}, z_k \}$

- one of the x_i must be true for the original clause to be true.

- it can be seen that if the original clause is satisfiable then its equivalent 3-sat representation is satisfiable too.

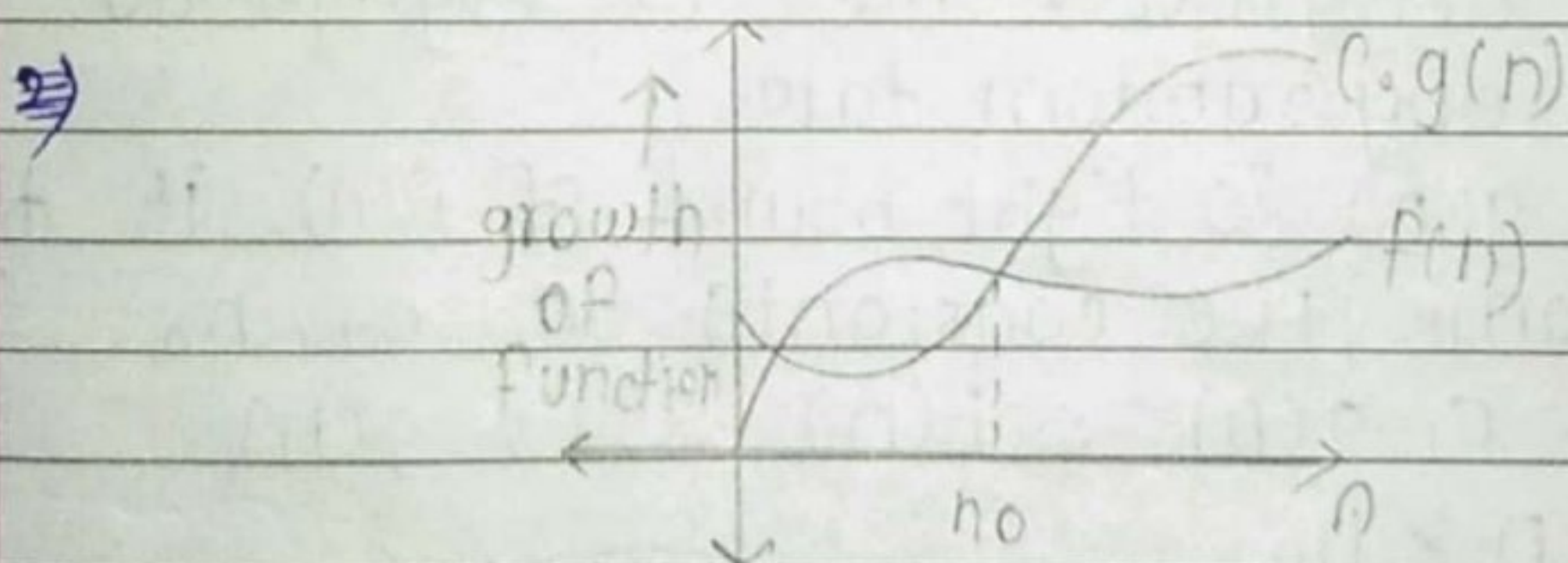
- Since any sat solution will satisfy the 3-sat instances and 3-sat solⁿ can set variables giving a sat solⁿ, the problems are equivalent.

- As SAT instances can be reducible to 3-SAT so 3-SAT is also NP hard.

Q.3 a) Define Big O, Omega and Theta Notation, state whether $100n^3 + 6n + 4 = O(n^3)$ is correct or not. Justify your answer.

→ 1) Big O :-

- (1) Big O notation defines upper bound for algorithm.
- (2) It means running time of algorithm cannot be more than its asymptotic upper bound for any input data.
- (3) Let $f(n)$ and $g(n)$ are two non negative functions indicating running time of 2 algorithms.
- (4) We say $g(n)$ is upper bound of $f(n)$ if there exist some +ve constants c & n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
- (5) It is denoted as $f(n) = O(g(n))$



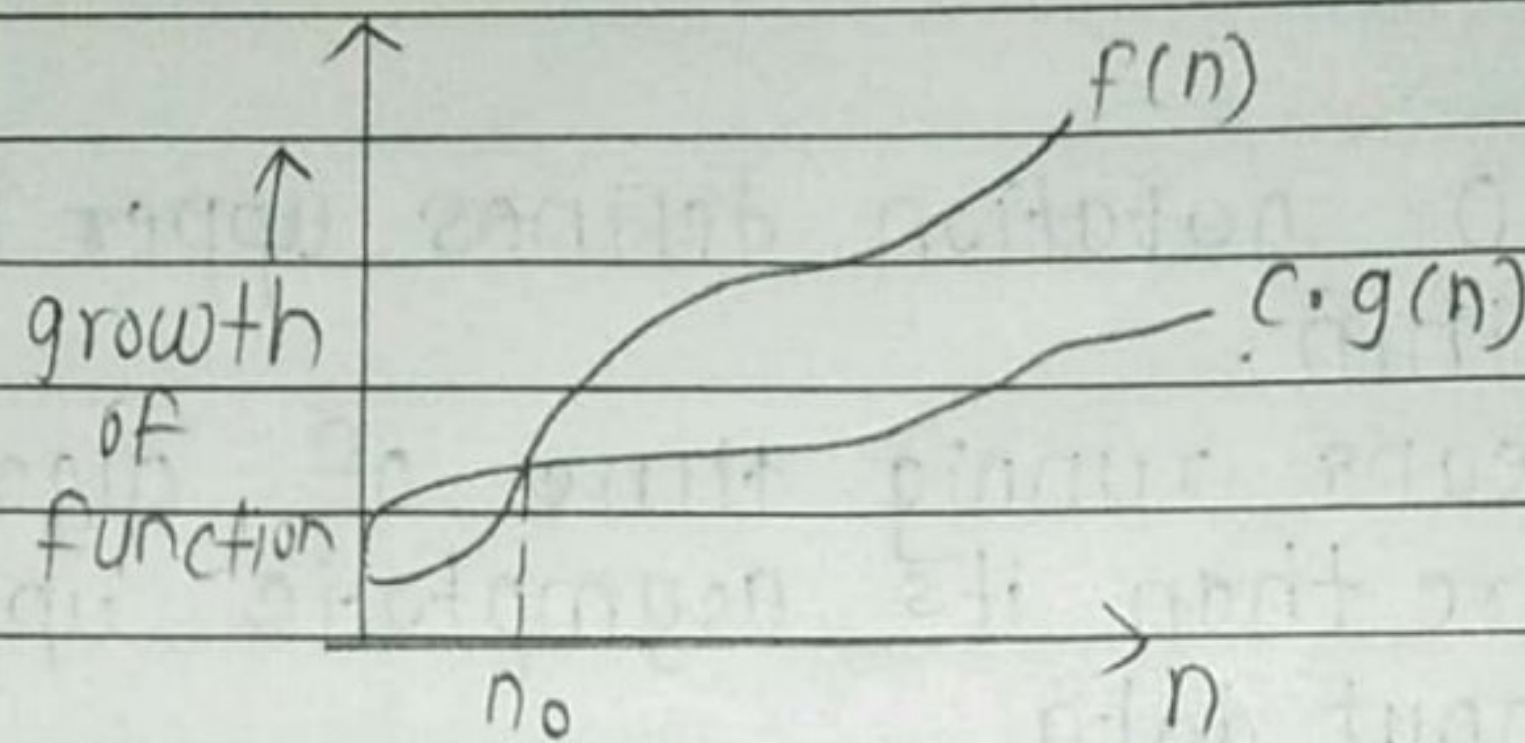
2) Big Omega (Ω) :-

- (1) Big Omega notation defines lower bound for the algorithm.
- (2) It means running time of algorithm cannot be less than its asymptotic lower bound for any random data.
- (3) We say $g(n)$ is lower bound of $f(n)$ if there exist some positive constants c & n_0 such that

$$0 \leq c \cdot g(n) \leq f(n)$$

for all $n \geq n_0$

(4) It is denoted as $f(n) = \Omega(g(n))$



3) Big theta (Θ) :-

(1) Big theta notation defines tight bound for the algorithm.

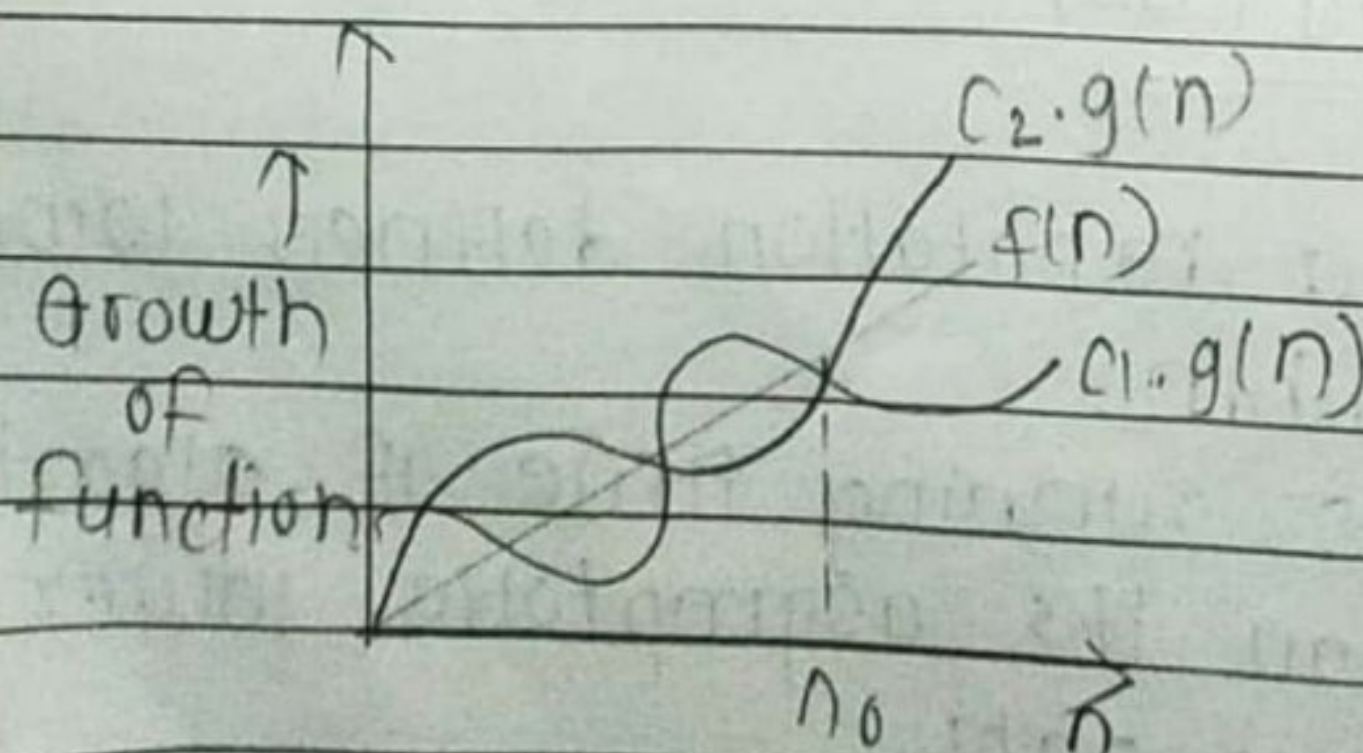
(2) It means running time of algorithm cannot be less than or greater than its asymptotic tight bound for any random data.

(3) we say $g(n)$ is tight bound of $f(n)$ if there exist some +ve constants c_1, c_2, n_0 such that

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

for all $n \geq n_0$

(4) It is denoted as $f(n) = \Theta(g(n))$



$$100n^3 + 6n + 4 = O(n^3)$$

Since big oh (O) notation is given we have to find upper bound of $f(n)$

$$\therefore f(n) = 100n^3 + 6n + 4$$

$$\therefore 0 \leq f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$

$$\therefore 0 \leq 100n^3 + 6n + 4 \leq c \cdot g(n)$$

$$0 \leq 100n^3 + 6n + 4 \leq 100n^3 + 6n^3 + 4n^3$$

$$0 \leq 100n^3 + 6n + 4 \leq 110n^3 \quad \text{for all } n \geq 1$$

$$\therefore c = 110, \quad g(n) = n^3, \quad n_0 = 1$$

$$\therefore f(n) = O(n^3)$$

Hence proved that $100n^3 + 6n + 4 = O(n^3)$ is correct.

Q. 4 A) Differentiate between NP hard and NP complete problem.

→ NP hard	NP complete
1. NP hard problem (say A) can be solved if and only if there is NP complete problem (say B) can be reducible into A in polynomial time	1. NP complete problems can be solved by deterministic algorithm in polynomial time.
2. To solve NP hard problems it must be in NP class	2. To solve NP complete problem it must be in both NP & NP hard problems.

3. NP hard problems are not decision problem.

3. NP complete problems are decision problems.

4. EX:-

Halting problems
Hamiltonian cycle problems

4. EX:-

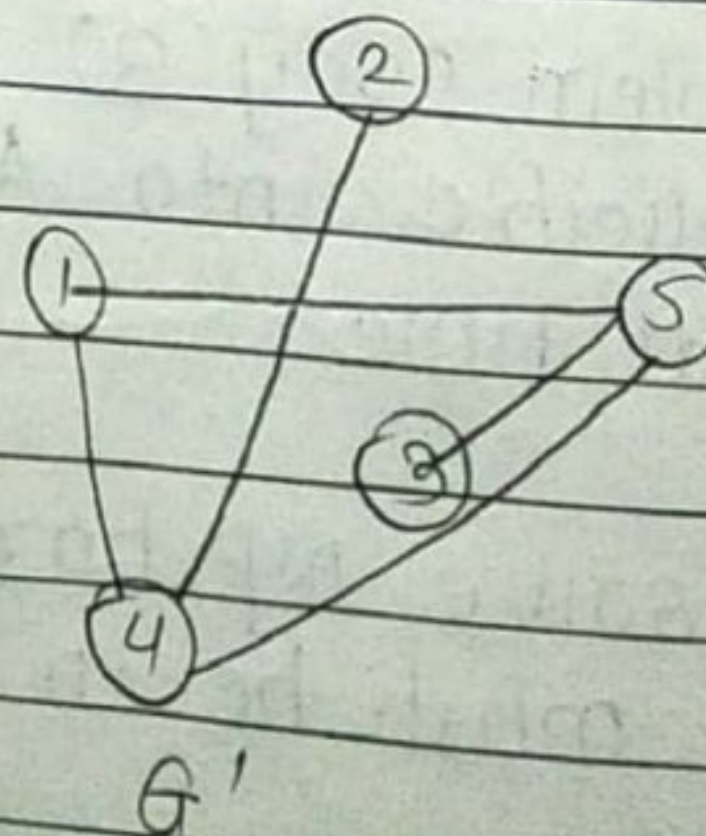
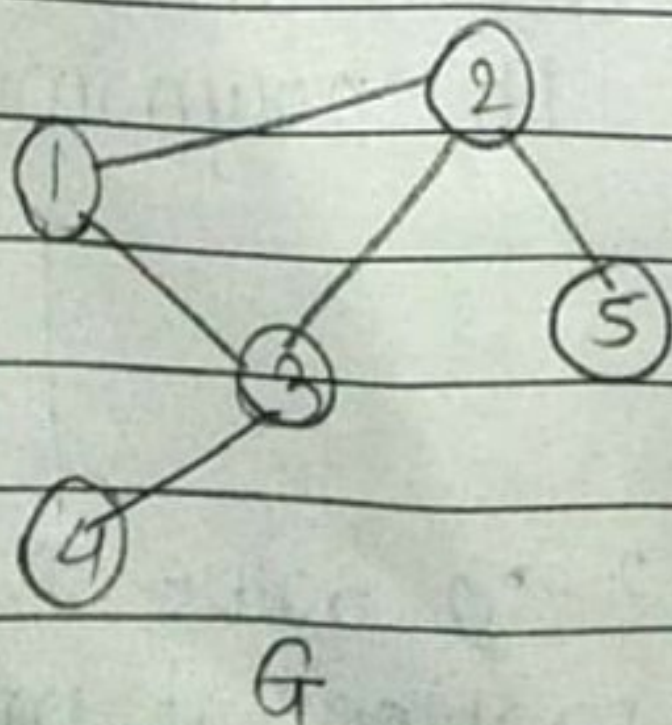
Determining whether a graph has hamiltonian cycle.

Q.4 B) prove that vertex cover problem is NP complete.

→ proof :-

1. let $G = (V, E)$ and k define an instance of clique decision problems (CDP)
2. Assume that $|V| = n$
3. We construct a graph G' such that G' has a node cover of size at most $n-k$ if & only if G has a clique of size at least k .
4. Graph G' is given by $G' = (V, \bar{E})$ where $\bar{E} = \{(u, v) \mid u \in V, v \in V \text{ and } (u, v) \notin E\}$
5. The set G' is known as complement of G .
6. G' can be obtained from G in polynomial time.

EX:-



- Fig. shows Graph G & its component G' . In this Fig G' has vertex cover of $\{4, 5\}$ since every edge of G' is incident either on node 4 or 5.
- Thus G has clique of size $5-2=3$ consisting of node 1, 2, 3.

7. Since CNF satisfiability (3 SAT) $\propto$ CDF and CDF $\propto$ Vertex cover.

We can say that vertex cover is np hard.

8. Vertex cover is also np because we can non deterministically choose subset $C \subseteq V$ of size k and verify in polynomial time that C is cover of G .

9. Since vertex cover is both np and np hard we can say that vertex cover is np complete.

Q.4 c) State whether the following functions are correct or incorrect and justify your answer.

→ i) $3n+2 = O(n)$

Since big oh (O) notation is given we have to find upper bound of $f(n)$

we will find c & n_0 such that

$$0 \leq f(n) \leq c \cdot g(n), \quad n \geq n_0$$

$$f(n) = 3n+2$$

$$\therefore 0 \leq 3n+2 \leq c \cdot g(n)$$

$$0 \leq 3n+2 \leq 3n+2n$$

$$0 \leq 3n+2 \leq 5n \quad \dots \text{for all } n \geq 1$$

$$\therefore c = 5, \quad n_0 = 1, \quad g(n) = n$$

$$\therefore f(n) = O(n)$$

$$\therefore 3n+2 = O(n) \text{ is correct.}$$

ii) $100n + 6 = o(n)$

$\therefore$ we have to find C & n_0 such that
 $0 \leq f(n) \leq C \cdot g(n)$ for all $n \geq n_0$

$f(n) = 100n + 6$

$\therefore 0 \leq 100n + 6 \leq C \cdot g(n)$

$0 \leq 100n + 6 \leq 100n + 6n$

$0 \leq 100n + 6 \leq 106n$ for all $n \geq 1$

$\therefore C = 106$, $g(n) = n$, $n_0 = 1$

$\therefore f(n) = o(n)$

$\therefore 100n + 6 = o(n)$ is correct.

iii) $10n^2 + 4n + 2 = o(n^2)$

$\therefore$ we have to find C & n_0 such that
 $0 \leq f(n) \leq C \cdot g(n)$... $n \geq n_0$

$f(n) = 10n^2 + 4n + 2$

$\therefore 0 \leq 10n^2 + 4n + 2 \leq C \cdot g(n)$

$0 \leq 10n^2 + 4n + 2 \leq 10n^2 + 4n^2 + 2n^2$

$0 \leq 10n^2 + 4n + 2 \leq 16n^2$ for all $n \geq 1$

$\therefore C = 16$, $g(n) = n^2$, $n_0 = 1$

$\therefore f(n) = o(n^2)$

$\therefore 10n^2 + 4n + 2 = o(n^2)$ is correct.